

Laborator 5

Directoare

5.1 Scopul lucrării

Laboratorul curent prezintă principalele apeluri de sistem în Linux pentru manipularea directoarelor.

5.2 Fundamente teoretice

5.2.1 Apeluri de sistem pentru lucrul cu directoare

```
#include <sys/types.h>
#include <dirent.h>

DIR *opendir(const char *name);
int closedir(DIR *dirp);
struct dirent *readdir(DIR *dirp);
void rewinddir(DIR *dirp);
```

5.2.1.1 opendir și closedir

Apelul de sistem **opendir** este utilizat pentru deschiderea unui director. Parametrul **name** reprezintă calea relativă sau absolută spre directorul care se dorește deschis. Rezultatul apelului este un pointer la un *directory stream*, de tip **DIR***. Acest stream are asociat un cursor, care ne permite să parcurgem directorul element cu element.

La deschiderea unui director, sistemul de operare alocă anumite structuri interne, care se vor elibera cu apelul de sistem **closedir**.

5.2.1.2 readdir

Pentru a citi următorul element dintr-un director se folosește apelul de sistem **readdir**, care primește ca parametru valoarea de tip **DIR*** returnată de **opendir**. Valoarea returnată este un pointer la o structură de tip **struct dirent**. Această structură poate să difere de la un sistem de operare la altul, dar standardul POSIX garantează că va conține cel puțin membrul **d_name**, indicând numele elementului.

5.2.1.3 rewinddir

Dacă dorim să mutăm cursorul asociat *directory stream*-ului la începutul unui director deschis, putem apela **rewinddir**, dând ca parametru stream-ul de tip **DIR*** returnat de **opendir**.

5.2.1.4 Exemplu

Dorim să scriem o funcție C care primește ca argument calea spre un director și îi listează conținutul.

```

1 | int listDir(const char *path)
2 | {
3 |     DIR *dir = NULL;
4 |     struct dirent *entry = NULL;
5 |
6 |     dir = opendir(path);
7 |     if(dir == NULL) {
8 |         perror("Could not open directory");
9 |         return -1;
10 |    }
11 |    while((entry = readdir(dir)) != NULL) {
12 |        printf("%s\n", entry->d_name);
13 |    }
14 |    closedir(dir);
15 |    return 0;
16 | }
```

Funcția începe prin a deschide directorul, la linia 6. În cazul în care deschiderea directorului a eșuat, `opendir` returnează `NULL`. În acest caz, vom afișa motivul erorii și vom ieși din funcție.

Citirea propriu-zisă a directorului se face în bucla de la liniile 11–13. Bucla apelează `readdir` la fiecare iterație, pentru a obține următorul element din director. Atunci când se ajunge la finalul directorului, `readdir` va returna `NULL`. După citire, structura `entry` va conține printre altele, numele fișierului în câmpul `d_name`, care se va afișa.

Înainte de finalul funcției, directorul trebuie închis, apelând `closedir` (linia 14).

5.2.2 Utilizarea link-urilor

Un link sau legătură este un tip de fișier care face legătura cu un alt fișier. Pe sisteme de tip UNIX (inclusiv Linux), avem două tipuri de link-uri:

- *hard link* - un fișier obișnuit asociat aceluiasi i-node. După ce este creat, un hardlink nu se mai deosebește de fișierul original. Atunci când ultima legătură se șterge, se șterge și i-node-ul, împreună cu conținutul.
- *symbolic link* - este un fișier special care indică locația altui fișier. Dacă fișierul original se șterge, symlink-ul devine invalid.

```

#include <unistd.h>

int link(const char *oldpath, const char *newpath);
int symlink(const char *target, const char *linkpath);
```

Apelul de sistem `link` este utilizat pentru a crea hardlink-uri, iar `symlink` pentru a crea link-uri simbolice. Ambele apeluri de sistem primesc ca prim argument calea spre un fișier existent, iar al doilea argument reprezintă calea spre link-ul ce se dorește a fi creat.

5.2.3 Extragerea de informații despre fișiere

```

#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>

int stat(const char *pathname, struct stat *statbuf);
```

```
int fstat(int fd, struct stat *statbuf);
int lstat(const char *pathname, struct stat *statbuf);
```

Pentru a obține metadate despre un fișier (sau director) putem folosi unul din apelurile de sistem `stat`, `fstat` sau `lstat`. Toate trei primesc în al doilea parametru un pointer la o structură de tipul `struct stat`, în care vor completa informații despre fișier. În caz de succes se va returna 0, altfel -1.

Apelurile `stat` și `lstat` primesc ca prim parametru numele sau calea fișierului, în timp ce `fstat` primește un descriptor de fișier.

Diferența dintre `stat` și `lstat` apare doar în cadrul legăturilor simbolice. Dacă avem un fișier `f1` care este de fapt o legătură simbolică spre fișierul `f2`, `stat` apelat pe `f1` va obține informațiile despre `f2`, în timp ce `lstat` va obține informațiile despre `f1`.

Printre metadatele prezente într-un `struct stat`, întâlnim:

- `off_t st_size` - pentru fișiere obișnuite, indică dimensiunea fișierului în octeți;
- `mode_t st_mode` - indică tipul fișierului și permisiunile.

Pentru a determina tipul unui fișier, putem aplica unul din macro-urile de mai jos pe câmpul `st_mode` dintr-un `struct stat`:

- `S_ISREG` - returnează o valoare nenulă dacă avem un fișier obișnuit (*regular*);
- `S_ISDIR` - returnează o valoare nenulă dacă avem un director;
- `S_ISLNK` - returnează o valoare nenulă dacă avem o legătură simbolică.

Tot cu ajutorul câmpului `st_mode` se pot identifica și permisiunile unui fișier. Mai exact, cei mai puțin semnificativi 9 biți ai acestui câmp se referă la permisiuni. Pentru a extrage doar acei biți, se poate aplica operația `&` (AND logic) cu masca `0777` (valoarea 777 în octal, care corespunde la 9 biți de 1).

5.2.3.1 Exemplu: afișarea tipului și a permisiunilor fișierelor

Vom extinde exemplul din secțiunea 5.2.1.4 prin afișarea tipului, pentru fiecare fișier listat.

```
1 ...
2 char filePath[512];
3 struct stat statbuf;
4 ...
5 while((entry = readdir(dir)) != NULL) {
6     snprintf(filePath, 512, "%s/%s", path, entry->d_name);
7     if(lstat(filePath, &statbuf) == 0) {
8         if(S_ISREG(statbuf.st_mode)) {
9             printf("[reg] ");
10        } else if(S_ISDIR(statbuf.st_mode)) {
11            printf("[dir] ");
12        } else if(S_ISLNK(statbuf.st_mode)) {
13            printf("[lnk] ");
14        } else {
15            printf("[unk] ");
16        }
17        printf("[%o] ", statbuf.st_mode & 0777);
18    } else {
19        printf("[err] ");
20    }
21    printf("%s\n", entry->d_name);
22 }
23 ...
```

Bucula de parcurgere a fișierelor (liniile 5–22) va începe prin construcția căii complete spre fișier (linia 6), prin concatenarea căii spre folderul curent cu separatorul / și cu numele fișierului curent. Pentru această operație vom utiliza funcția `snprintf`, care “afișează” date în mod similar cu `printf`, dar într-un string. Preferăm să folosim `snprintf` în loc de `sprintf`, deoarece prima variantă primește ca argument și lungimea string-ului destinație și garantează că nu o va depăși.

Apelul de sistem `lstat` de la linia 7 are nevoie de calea completă spre fișier și nu doar de numele acestuia. De exemplu, dacă vrem să aflăm informații despre fișierul `f1`, care se află în folderul `d1/d2` (cale relativă la folderul curent) și apelăm `lstat("f1", ...)` în loc de `lstat("d1/d2/f1", ...)`, se vor căuta metadate despre `f1` în folderul curent, în loc să se caute în `d1/d2`.

Dacă `lstat` s-a executat cu succes, va întoarce valoarea 0, caz în care ne putem baza că s-a completat câmpul `st_mode` din structura `statbuf` transmisă ca pointer. La liniile 8, 10 și 12 se folosesc macro-urile prezentate în secțiunea anterioară pentru a decide dacă fișierul este unul obișnuit, un director, sau respectiv o legătură simbolică. Cele trei tipuri nu sunt exhaustive, există și alte tipuri posibile pentru un fișier.

La linia 17 afișăm permisiunile fișierului, aplicând masca de biți 0777 pe câmpul `st_mode`. Rezultatul se afișează folosind specificatorul `%o` (octal).

5.2.3.2 Exemplu: listarea recursivă a unui folder

În exemplele anterioare s-a afișat doar conținutul direct al unui folder, fără a se intra în subfoldere. În acest exemplu vom lista recursiv fiecare subfolder al folderului dat.

```

1 void listRec(const char *path)
2 {
3     DIR *dir = NULL;
4     struct dirent *entry = NULL;
5     char fullPath[512];
6     struct stat statbuf;
7
8     dir = opendir(path);
9     if(dir == NULL) {
10         perror("Could not open directory");
11         return;
12     }
13     while((entry = readdir(dir)) != NULL) {
14         if(strcmp(entry->d_name, ".") != 0
15            && strcmp(entry->d_name, "..") != 0) {
16             snprintf(fullPath, 512, "%s/%s", path, entry->d_name);
17             if(lstat(fullPath, &statbuf) == 0) {
18                 printf("%s\n", fullPath);
19                 if(S_ISDIR(statbuf.st_mode)) {
20                     listRec(fullPath);
21                 }
22             }
23         }
24     }
25     closedir(dir);
26 }
```

Parcurgerea recursivă a unui director are aceeași structură ca și parcurgerea obișnuită: se deschide directorul (linia 8), se citesc intrările într-o buclă (liniile 13–24), iar la final se închide (linia 25).

O diferență importantă este tratarea subfolderelor speciale `.` și `..` aflate implicit în orice folder. Aceste subfoldere ne duc la folderul curent, respectiv la folderul părinte. Dacă am apela recursiv și pe aceste subfoldere, algoritmul de parcurgere nu s-ar mai termina. Din acest motiv, în fiecare iterație a buclei se verifică dacă numele elementului curent e diferit de `.` și de `..` (liniile 14–15). Pentru simplitate s-a folosit funcția `strcmp`, care returnează 0 dacă două string-uri sunt egale.

Iterația continuă cu construcția căii complete spre fișier (linia 16), din motivul explicat în secțiunea 5.2.3.1. Această cale va fi dată ca parametru apelului de sistem `lstat` (linia 17) și va fi afișată (linia 18).

Pentru a lista și conținutul subfolderelor, fiecare intrare trebuie verificată dacă este folder sau nu (linia 19). În caz că este folder, pur și simplu vom apela recursiv funcția `listRec()` pe calea completă spre subfolder.

Observație: Exemplul de mai sus va lista doar elementele a căror cale nu depășește 512 caractere.

5.3 Mersul lucrării

5.3.1 Întrebări recapitulative

1. Ce rol au elementele "." și ".." din fiecare director?
2. Ce funcții se folosesc pentru a citi conținutul unui director? Dar pentru a-i scrie conținutul?
3. Cu ce apel de sistem pentru operații pe fișiere este echivalent `rewinddir`?
4. Care este diferența dintre un *hard link* și un *symbolic link*?
5. Dându-se o cale în sistemul de fișiere, cum aflăm dacă elementul respectiv este un fișier sau director?
6. În ce câmp dintr-o structură de tip `struct stat` se găsesc permisiunile unui fișier?

5.3.2 Probleme rezolvate

1. (*l05p1.list.c*) Să se scrie un program care primește ca argument în linia de comandă calea spre un folder și listează conținutul acestuia, afișând pentru fiecare element tipul acestuia (fișier, folder sau legătură simbolică), respectiv permisiunile. Programul este descris în secțiunile 5.2.1.4 și 5.2.3.1.
2. (*l05p2.list-rec.c*) Implementați un program care primește ca argument în linia de comandă calea spre un director și afișează recursiv conținutul acestuia. Programul este descris în secțiunea 5.2.3.2.

5.3.3 Probleme propuse

1. (35p) Implementați funcția `off_t dirSize(const char* dirPath)` care primește ca argument calea spre un director și returnează dimensiunea acestuia, ca suma tuturor fișierelor obișnuite din acesta, inclusiv cele din subfoldere. Dimensiunea unui symlink se consideră 0. *Bonus (20p):* Dacă printre fișierele din director se află mai multe hard link-uri către același i-node, dimensiunea acestuia se va aduna o singură dată.
2. (35p) Scrieți un program care primește ca argument în linia de comandă calea spre un folder, numele unui fișier și un string. Programul va căuta recursiv în folder fișierele cu numele dat, ce conțin string-ul primit ca al treilea argument. Pentru fișierele găsite se vor crea legături simbolice în folderul curent, adăugând o extensie incrementală fiecărei legături (prima legătură va avea `.1` la final, a doua `.2`, ș.a.m.d.).
Observație: Pentru punctaj parțial se poate implementa doar căutarea fișierelor după nume, fără a mai căuta string-ul dat în conținut.
3. (35p) Scrieți un program care primește ca argument numele unui folder și îl șterge împreună cu tot conținutul acestuia.
Indicații: Consultați paginile de manual pentru apelurile de sistem `unlink` (pentru ștergerea unui fișier) și `rmdir` pentru ștergerea unui director gol.

